# NETFLIX CONTENT ANALYSIS

Python, Pandas, Data Visualization & MySQL

| | |
|---|---|
| **Project Type** | Data Analytics Mini Project |
| **Prepared by** | Harshavardhan |
| **Dataset** | Netflix Movies and TV Shows (Kaggle) |
| **Dataset size** | 8,804 records and 12 columns |
| **Tools** | Python, Pandas, NumPy, Matplotlib, Seaborn, MySQL Workbench |

A practical data analytics project to explore Netflix's content catalogue, clean and summarize data using Python, visualize patterns, and answer business questions using SQL.

# Abstract

This project explores the Netflix Movies and TV Shows catalogue using a combined Python and SQL workflow. The dataset contains 8,804 titles and 12 fields describing title identifiers, content type, title, director, cast, country, date added, release year, rating, duration, genre/category labels, and description. Python was used for data inspection, cleaning, exploratory analysis, and chart creation. MySQL was used to query the imported data and answer practical questions about content types, countries, ratings, movie duration, additions by year, release-year patterns, and genres.

The purpose is to demonstrate an end-to-end analytics workflow: understand the data, prepare it for analysis, calculate useful summaries, communicate findings, and document the work in a reproducible format.

# Contents

• 1. Problem Statement and Objectives

• 2. Dataset Overview

• 3. Tools and Technologies

• 4. Methodology

• 5. Python Analysis and Findings

• 6. SQL Analysis

• 7. Key Insights and Business Interpretation

• 8. Limitations and Recommendations

• 9. Conclusion

• Appendix A: SQL Query Collection

# 1. Problem Statement and Objectives

**Problem statement:** Streaming platforms offer large catalogues of movies and television shows. A structured analysis can help summarize catalogue composition, identify common genres and ratings, examine release and addition trends, and understand movie duration.

**Project objectives**

• Inspect the dataset's dimensions, columns, data types, missing values, and duplicate records.

• Clean and prepare data for reliable exploratory analysis.

• Summarize Movies and TV Shows and investigate ratings, countries, genres, release years, and durations.

• Create visualizations that communicate the most useful patterns.

• Load the data into MySQL and answer similar questions with SQL.

• Present the workflow and findings in a clear portfolio-style report.

## Questions explored

• How many Movies and TV Shows are present?

• Which countries and content ratings occur most often?

• What is the typical movie duration?

• In which years was the largest volume of content added?

• How do Movies and TV Shows vary by release year?

• Which genres/categories appear most frequently?

# 2. Dataset Overview

The project uses the Netflix Movies and TV Shows dataset commonly distributed through Kaggle. The imported dataset was confirmed to contain 8,804 rows. The working schema contains 12 columns.

| Column | Meaning |
|---|---|
| show_id | Unique title identifier |
| type | Movie or TV Show |
| title | Title name |
| director | Director or directors |
| cast | Cast members |
| country | Country or countries associated with the title |
| date_added | Date the title was added to Netflix |
| release_year | Original release year |
| rating | Content rating |
| duration | Movie runtime or number of TV seasons |
| listed_in | Genres/categories associated with the title |
| description | Short title description |

**Important data note:** the country field may contain multiple countries in one cell, and the listed_in field may contain multiple genres separated by commas. Counts by exact text value therefore do not always represent counts by individual country or genre unless the values are split first.

# 3. Tools and Technologies

• **Python:** main programming language for data inspection and analysis.

• **Pandas:** tabular data loading, cleaning, transformation, grouping, and summarization.

• **NumPy:** numerical operations where required.

• **Matplotlib and Seaborn:** visual exploration and chart creation.

• **MySQL / MySQL Workbench:** relational storage, SQL querying, aggregation, and analysis.

• **Jupyter Notebook:** iterative analysis and documentation.

• **GitHub:** project version control and portfolio sharing.

# 4. Methodology

## 4.1 Data loading and inspection

The CSV file was loaded into Pandas. The initial inspection covered dataset shape, column names, data types, sample records, missing values, and duplicate rows. The dataset had 8,804 rows and 12 columns.

## 4.2 Data cleaning

• Duplicate records were checked; the project audit reported no duplicate rows.

• Missing values were reviewed by column.

• Missing values in categorical/text fields were represented as 'Unknown' where appropriate for analysis.

• The date_added field had 10 missing values; these were handled as missing rather than inventing dates.

• date_added was converted to a datetime type in Python.

• Movie duration was extracted as a numeric number of minutes for duration-specific analysis.

## 4.3 Exploratory analysis

The analysis used frequency counts, grouped summaries, numeric summaries, and charts to explore catalogue composition. The same dataset was loaded into MySQL to reproduce and extend the analysis with SQL.

## 4.4 SQL import

A MySQL database named **netflix_analysis** and a table named **netflix_titles** were created. The CSV was imported using LOAD DATA LOCAL INFILE after enabling LOCAL INFILE for the connection. The row-count check returned 8,804 records.

# 5. Python Analysis and Findings

## 5.1 Content type distribution

The Python summary showed 6,128 Movies and 2,676 TV Shows.

| Content type | Titles | Approx. share |
|---|---|---|
| Movie | 6,128 | 69.6% |

| | | |
|---|---|---|
| TV Show | 2,676 | 30.4% |
| Total | 8,804 | 100% |

Interpretation: Movies represent roughly seven out of every ten catalogue records, so the catalogue is more movie-heavy than TV-show-heavy.

## 5.2 Most frequent genre/category labels

| Genre/category | Count |
|---|---|
| International Movies | 2,752 |
| Dramas | 2,427 |
| Comedies | 1,674 |
| International TV Shows | 1,351 |
| Documentaries | 866 |
| Action & Adventure | 859 |
| TV Dramas | 763 |
| Independent Movies | 756 |
| Children & Family Movies | 641 |
| Romantic Movies | 616 |

These counts reflect the project's genre/category frequency summary. Because a title can have multiple category labels, the categories overlap and their counts should not be added together as if each title belonged to exactly one genre.

## 5.3 Country labels

| Country label | Count |
|---|---|
| United States | 3,686 |
| India | 1,046 |
| Unknown | 831 |
| United Kingdom | 803 |
| Canada | 445 |
| France | 393 |
| Japan | 318 |
| Spain | 232 |
| South Korea | 231 |
| Germany | 226 |

The United States is the most frequent country label in the Python summary, followed by India and the United Kingdom. Since some records contain more than one country, a normalized country-level analysis would require splitting multi-country values.

## 5.4 Content ratings

The most frequent rating labels in the Python summary were TV-MA (3,207), TV-14 (2,158), TV-PG (862), R (799), and PG-13 (490). The presence of both TV-specific and film-style ratings reflects the mix of Movies and TV Shows in the catalogue.

### 5.5 Release year, duration, and date-added trends

The project created a release-year trend chart, a movie-duration histogram, and a date-added analysis. Movie durations were extracted from values such as '90 min' to support numeric analysis. Date-added analysis was kept distinct from release-year analysis: the former measures when a title was added to Netflix, while the latter records the title's release year.

The project notebook also included visualizations using Matplotlib/Seaborn. Charts were saved in the visualizations folder for use in the project repository.

# 6. SQL Analysis

The MySQL portion reproduced key exploratory questions using SELECT, WHERE, GROUP BY, ORDER BY, aggregate functions, conditional aggregation, date parsing, and a recursive common table expression (CTE).

• **Content type totals:** GROUP BY type and COUNT(*) to count Movies and TV Shows.

• **Type percentages:** Calculate each type's share of all 8,804 records.

• **Country labels:** Rank exact country field values; multi-country cells need normalization for per-country counts.

• **Ratings:** Count records by content rating.

• **Movie duration:** Extract numeric minutes from duration and calculate average, minimum, and maximum.

• **Titles added by year:** Parse date_added and group by year.

• **Release year by content type:** Use SUM(CASE WHEN...) to count Movies and TV Shows by release year.

• **Genre/category labels:** Split comma-separated labels using a recursive CTE, then count each label.

• **Rating by content type:** Use conditional aggregation to compare Movies and TV Shows by rating.

### Database structure

```
CREATE DATABASE netflix_analysis;
USE netflix_analysis;

CREATE TABLE netflix_titles (
    show_id VARCHAR(10) PRIMARY KEY,
    type VARCHAR(20),
    title VARCHAR(255),
    director TEXT,
    cast TEXT,
    country TEXT,
    date_added VARCHAR(50),
    release_year INT,
    rating VARCHAR(20),
    duration VARCHAR(30),
    listed_in TEXT,
    description TEXT
);
```

The date_added field was stored as text in the SQL table and parsed at query time with STR_TO_DATE. This makes the import straightforward, but a production-grade database could store date_added as a DATE column after validating all source values.

# 7. Key Insights and Business Interpretation

### Movies dominate the catalogue

6,128 of 8,804 records are Movies, approximately 69.6%; TV Shows make up about 30.4%.

### International and drama labels are prominent

International Movies and Dramas were the top category labels in the Python summary.

### United States is the leading country label

The United States had the largest country count in the summary, with India next.

### Mature ratings are common

TV-MA and TV-14 were the two most common rating labels in the Python summary.

### Release and addition years answer different questions

Release year describes when content was released; date_added describes when it was added to the Netflix catalogue.

### Multi-value columns require care

Country and genre values may contain comma-separated items. Counting exact strings can undercount individual entities or treat combinations as separate labels.

## 8. Limitations and Recommendations

• This dataset is a snapshot of a Netflix catalogue, not a live inventory. Results should not be interpreted as current availability.

• The analysis describes the dataset and does not establish why Netflix made particular content decisions.

• Country and genre columns can contain multiple values. For more accurate entity-level analysis, normalize these into separate rows or bridge tables.

• Missing or unknown values can affect counts and should be reported transparently.

• The date-added SQL query depends on the source date format; validate parsing results and exclude invalid dates.

• A useful extension would be to create a Power BI or Streamlit dashboard using the cleaned dataset and the SQL summaries.

• For a polished repository, include the notebook, saved visualizations, requirements.txt, README.md, and the SQL script in a clearly named sql folder.

## 9. Conclusion

The Netflix Content Analysis mini project demonstrates a practical end-to-end analytics workflow using Python and MySQL. The Python phase covered inspection, cleaning, summary statistics, and visual exploration. The SQL phase translated the analysis into database queries and introduced techniques such as conditional aggregation and recursive CTEs. The results indicate a catalogue dominated by Movies, with International Movies and Dramas among the most common category labels, the United States as the most frequent country label, and TV-MA/TV-14 as common ratings.

The project provides a foundation for further work in data analytics: data cleaning, exploratory analysis, SQL querying, visual communication, and documentation. The findings should be read with the dataset's limitations in mind, especially its snapshot nature and multi-value country/genre fields.

# Appendix A: SQL Query Collection

The following queries capture the principal SQL analyses completed during the project. Execute them after selecting the netflix_analysis database.

## A1. Total number of titles

```
SELECT COUNT(*) AS total_titles
FROM netflix_titles;
```

## A2. Movies vs TV Shows and percentage

```
SELECT
    type,
    COUNT(*) AS total_titles,
    ROUND(COUNT(*) * 100.0 /
        (SELECT COUNT(*) FROM netflix_titles), 2) AS percentage
FROM netflix_titles
GROUP BY type
ORDER BY total_titles DESC;
```

## A3. Top 10 exact country labels

```
SELECT country, COUNT(*) AS total_titles
FROM netflix_titles
WHERE country IS NOT NULL AND TRIM(country) <> ''
GROUP BY country
ORDER BY total_titles DESC
LIMIT 10;
```

## A4. Content rating distribution

```
SELECT rating, COUNT(*) AS total_titles
FROM netflix_titles
WHERE rating IS NOT NULL AND TRIM(rating) <> ''
GROUP BY rating
ORDER BY total_titles DESC;
```

## A5. Movie duration statistics

```
SELECT
    COUNT(*) AS total_movies,
    ROUND(AVG(CAST(SUBSTRING_INDEX(duration, ' ', 1)
        AS UNSIGNED)), 2) AS average_duration_minutes,
    MIN(CAST(SUBSTRING_INDEX(duration, ' ', 1)
        AS UNSIGNED)) AS shortest_movie_minutes,
    MAX(CAST(SUBSTRING_INDEX(duration, ' ', 1)
        AS UNSIGNED)) AS longest_movie_minutes
FROM netflix_titles
WHERE type = 'Movie'
  AND duration REGEXP '^[0-9]+ min$';
```

## A6. Titles added by year

```
SELECT
    YEAR(STR_TO_DATE(date_added, '%M %e, %Y')) AS added_year,
    COUNT(*) AS total_titles
FROM netflix_titles
WHERE date_added IS NOT NULL AND TRIM(date_added) <> ''
GROUP BY added_year
HAVING added_year IS NOT NULL
ORDER BY total_titles DESC;
```

## A7. Movies and TV Shows by release year

```
SELECT
    release_year,
    SUM(CASE WHEN type = 'Movie' THEN 1 ELSE 0 END) AS total_movies,
    SUM(CASE WHEN type = 'TV Show' THEN 1 ELSE 0 END) AS total_tv_shows,
    COUNT(*) AS total_titles
FROM netflix_titles
WHERE release_year IS NOT NULL
GROUP BY release_year
ORDER BY release_year;
```

## A8. Top 10 individual genre labels using a recursive CTE

```
WITH RECURSIVE genre_split AS (
    SELECT
        show_id,
        TRIM(SUBSTRING_INDEX(listed_in, ',', 1)) AS genre,
        CASE
            WHEN listed_in LIKE '%,%'
            THEN SUBSTRING(listed_in, LOCATE(',', listed_in) + 1)
            ELSE NULL
        END AS remaining_genres
    FROM netflix_titles
    WHERE listed_in IS NOT NULL AND TRIM(listed_in) <> ''

    UNION ALL

    SELECT
        show_id,
        TRIM(SUBSTRING_INDEX(remaining_genres, ',', 1)),
        CASE
            WHEN remaining_genres LIKE '%,%'
            THEN SUBSTRING(remaining_genres, LOCATE(',', remaining_genres) + 1)
            ELSE NULL
        END
    FROM genre_split
    WHERE remaining_genres IS NOT NULL
)
SELECT genre, COUNT(*) AS total_titles
FROM genre_split
WHERE genre <> ''
GROUP BY genre
ORDER BY total_titles DESC
LIMIT 10;
```

## A9. Ratings by content type

```
SELECT
    rating,
    SUM(CASE WHEN type = 'Movie' THEN 1 ELSE 0 END) AS total_movies,
    SUM(CASE WHEN type = 'TV Show' THEN 1 ELSE 0 END) AS total_tv_shows,
    COUNT(*) AS total_titles
FROM netflix_titles
WHERE rating IS NOT NULL AND TRIM(rating) <> ''
GROUP BY rating
ORDER BY total_titles DESC;
```

Note: The recursive genre query requires a MySQL version that supports recursive CTEs (MySQL 8.0+). Country results from A3 are exact country-field values, not fully split individual-country counts.